# Web-приложения и их архитектура JS. Асинхронность.

# Введение

JavaScript — это синхронный и однопоточный язык программирования.

Что означает синхронность? Допустим, у нас есть две строки кода. Первая выполняется после второй. Синхронность означает, что вторая строка не может начать выполнение, пока первая строка не завершит свою работу.

Что означает однопоточность? Здесь всё ещё проще. Однопоточность в JavaScript означает, что одновременно может выполняться только один блок кода.

На этом этапе должен возникнуть вопрос: как в этом случае достигается асинхронность в JavaScript? В конце концов, мы знаем, что в наших программах некоторые асинхронные операции (setTimeout, HTTP-запросы и т. д.) довольно распространены.

# Call Stack & Event Loop

# Call Stack

В JavaScript стек вызовов — это механизм, используемый движком (например, V8) для отслеживания своего места в скрипте, вызывающем несколько функций. Он следует принципу «последний вошел — первый вышел» (LIFO), то есть последняя добавленная функция выполняется первой и удаляется первой.

Как это работает

- Помещение в стек: При вызове функции движок создает новый контекст выполнения (или кадр стека) и «помещает» его на вершину стека.
- Выполнение: Движок выполняет функцию, находящуюся в данный момент на вершине стека. Если эта функция вызывает другую функцию, новая функция помещается на вершину, приостанавливая предыдущую.
- Извлечение из стека: Как только функция завершается (возвращает значение или заканчивается), она «извлекается» из стека, и выполнение возобновляется с того места, где оно было прервано, в функции ниже.

# Основные характеристики

- Однопоточность: JavaScript имеет только один стек вызовов, то есть он может выполнять только одну операцию за раз.
- Глобальный контекст выполнения: При запуске скрипта сначала в стек помещается глобальный контекст (часто обозначаемый как main или (anonymous)).
- Переполнение стека: Стек имеет фиксированный размер (обычно 10 000–15 000 кадров). Если функция вызывает саму себя бесконечно (бесконечная рекурсия), она превышает этот предел и выдает ошибку RangeError: Maximum call stack size exceeded.

# Проблема синхронности в JS

В этом блоке кода команды выполняются одна за другой:

1. Получаем ссылку на элемент <button>, который уже есть в DOM.
2. Добавляем к кнопке обработчик события click так, что при нажатии на неё:
   1. Выводится сообщение alert().
   2. После закрытия сообщения создаём элемент <p> (абзац).
   3. Затем добавляем в абзац текст.
   4. В конце добавляем созданный абзац в тело документа.

```
const btn = document.querySelector("button");
btn.addEventListener("click", () => {
  alert("You clicked me!");

  let pElem = document.createElement("p");
  pElem.textContent = "This is a newly-added paragraph.";
  document.body.appendChild(pElem);
});
```

# Проблема синхронности в JS

Пока выполняется каждая операция, ничего больше не может произойти — обработка (отображение) документа приостановлена. Так происходит, потому что JavaScript является однопоточным. В каждый момент времени может выполняться только одна команда, обрабатываемая в единственном — главном потоке. Все остальные действия блокируются до окончания выполнения текущей команды.

# Асинхронность

По причинам, упомянутым ранее (например, относящимся к блокировке), множество Web API особенностей теперь используют асинхронный код, особенно те,что имеют доступ к внешним устройствам или получают от них некоторые ресурсы, такие как получение файла из сети, запрос к базе данных и получение данных из базы, доступ к потоковому видео на веб-камере, просмотр дисплея на гарнитуре виртуальной реальности.

# Примеры асинхронных операций

```
1   console.log('111');
2
3   setTimeout(function() {
4       console.log('222');
5   }, 3000);
6
7   console.log('333');
```

```
1   console.log('1111');
2
3   setTimeout(function() {
4       console.log('2222');
5   }, 0);
6
7   console.log('3333');
```

# Event Loop

Event Loop (цикл событий) — это механизм, позволяющий JavaScript (и Node.js) выполнять асинхронные неблокирующие операции в однопоточном режиме. Он постоянно проверяет стек вызовов: если тот пуст, Event Loop берет задачи из очередей (микротаски или макротаски) и передает их на выполнение, обеспечивая отзывчивость интерфейсов.

Как работает Event Loop:

- Синхронный код: Выполняется в первую очередь в Call Stack (стеке вызовов).
- Асинхронный код: Отправляется в Web API (браузер) или API Node.js (таймеры, сеть, I/O).
- Очереди задач: Когда асинхронная операция завершается, её callback помещается в очередь.
- Микротаски (Microtasks): Promise (.then/catch), queueMicrotask, MutationObserver. Выполняются немедленно после текущего синхронного кода, приоритет выше.
- Макротаски (Macrotasks/Tasks): setTimeout, setInterval, события DOM.
- Цикл: Event Loop берет все микротаски, затем одну макротаску, снова микротаски — и так до бесконечности.

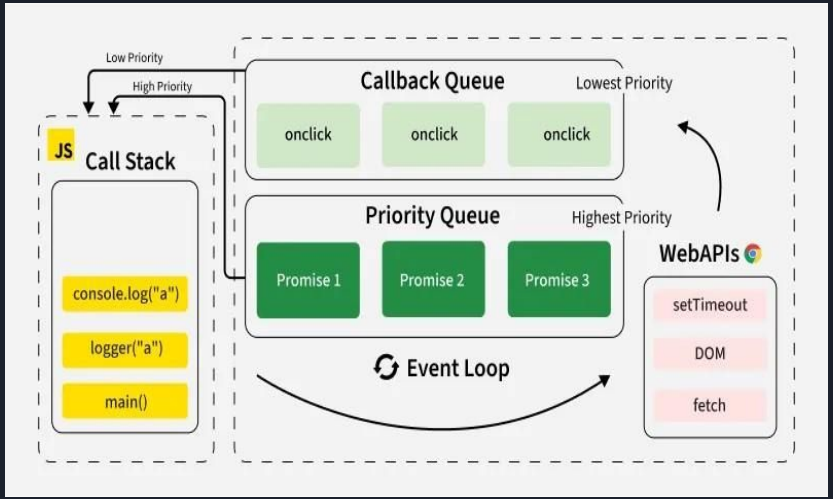
# Event Loop

На самом деле, цикл событий может включать в себя множество различных инструментов. Все зависит от среды, в которой мы запускаем наш код. Сегодня мы рассмотрим цикл событий исключительно в браузере (и даже тогда не полностью, мы не будем касаться рендеринга).

Стек — это наш стек вызовов.

Веб-API — это набор готовых инструментов, предоставляемых средой браузера.

Очередь обратных вызовов (или очередь задач) — очередь задач. Здесь хранятся задачи, готовые к выполнению.



# Пример асинхронного кода

```
1   console.log('111');
2
3   setTimeout(() => {
4       console.log('222');
5
6       setTimeout(() => console.log('333'), 0);
7   }, 0);
8
9   setTimeout(() => console.log('444'));
10
11  console.log('555');
```

# Работа с асинхронным кодом

В приведенных выше примерах вы можете увидеть один из подходов к работе с асинхронным кодом в JavaScript. Это подход с использованием функций обратного вызова. Он работает очень просто: мы вызываем setTimeout, делаем HTTP-запрос или даже прикрепляем обработчик к событию через addEventListener, а затем передаем функцию обратного вызова, которая будет вызвана по истечении таймера / когда запрос вернет ответ / произойдет событие.

Долгое время этот подход был, по сути, единственным возможным способом работы с асинхронными операциями. Но у него были проблемы. Пока операций было мало или они происходили редко, все выглядело и работало хорошо. Однако, когда появлялись цепочки вызовов, такие как: запрос на получение данных с сервера, в функции обратного вызова обработчика новый запрос к другому источнику, где в обработчике есть еще один запрос к третьему источнику и т. д., код становился очень громоздким, менее читаемым и буквально разрастался с невероятной скоростью. Такая вложенность значительно смещала код вправо на экране. Это то, что часто называют «адом обратных вызовов».

Нужен был выход из этой ситуации. И оно было найдено.

# Callback hell

```
function hell(win) {
 // for listener purpose
 return function() {
   loadLink(win, REMOTE_SRC+'/assets/css/style.css', function() {
     loadLink(win, REMOTE_SRC+'/lib/async.js', function() {
       loadLink(win, REMOTE_SRC+'/lib/easyXDM.js', function() {
         loadLink(win, REMOTE_SRC+'/lib/json2.js', function() {
           loadLink(win, REMOTE_SRC+'/lib/underscode.min.js', function() {
             loadLink(win, REMOTE_SRC+'/lib/backbone.min.js', function() {
               loadLink(win, REMOTE_SRC+'/dev/base_dev.js', function() {
                 loadLink(win, REMOTE_SRC+'/assets/js/deps.js', function() {
                   loadLink(win, REMOTE_SRC+'/src/' + win.loader_path + '/loader.js', function() {
                     async.eachSeries(SCRIPTS, function(src, callback) {
                       loadScript(win, BASE_URL+src, callback);
                     });
                   });
                 });
               });
             });
           });
         });
       });
     });
   });
 };
}
```

# Promise, async/await

# Promise

Промисы стали отличным решением всех проблем. С их появлением исчезла удручающая ситуация с разрастающимся кодом, и на смену ей пришла очень красивая цепочка вызовов.

Промис — это объект, представляющий асинхронную задачу и результат её выполнения. Он может находиться в одном из трёх состояний: ожидание (pending), успешное завершение (fulfilled), завершение с ошибкой (rejected).

```
1   function getUsers() {
2       return new Promise((resolve, reject) => {
3           asyncronouslyGetUsers((err, users) => {
4               if (err) {
5                   return reject(err); // return не обязателен
6               }
7
8               resolve(users);
9           });
10      });
11  }
```

# Promise API

Экземпляр промиса содержит три метода:

- .then(callbackOnResolve, callbackOnReject);
- .catch(callbackOnReject);
- .finally(callback).

Используя эти методы, строится цепочка вызовов. В случае, когда необходимо обработать успешный результат предыдущей операции, используется then. Когда необходимо обработать неудачный сценарий, используется catch. Когда необходимо выполнить какое-либо действие третьей стороны, независимо от ситуации, используется finally.

**Метод then также можно использовать для обработки неудачных сценариев. Однако это не рекомендуется, поскольку... catch лучше подходит для этой задачи.

# Promise API

Помимо методов экземпляров промисов, существует также ряд статических методов объекта Promise, которые помогают в работе с ними:

- Promise.all([promise1, value, ..., promiseN]); — запускает несколько промисов параллельно, возвращая результат в виде массива результатов всех промисов или первой обнаруженной ошибки.
- Promise.race([promise1, value, ..., promiseN]); — запускает несколько промисов параллельно, возвращая результат первого выполненного промиса или первую обнаруженную ошибку.
- Promise.allSettled([promise1, value, ..., promiseN]); — запускает несколько промисов параллельно, возвращая результат в виде массива объектов вида: { status: “fulfilled”, value: result } или { status: “rejected”, reason: error }.
- Promise.any([promise1, value, ..., promiseN]); — запускает несколько промисов параллельно, возвращая результат первого выполненного промиса.

# Макротаски

В цикле событий есть две очереди задач: макрозадачи (задачи/макротаски) и микрозадачи (задания/микрозадачи).

Макрозадачи выполняются по очереди, по одной за раз (т.е. за итерацию цикла событий). Примерами макрозадач являются функции обратного вызова, передаваемые в качестве параметров функции setTimeout, обработка событий браузера, XMLHttpRequest или fetch. Каждая из этих задач является асинхронной и будет добавлена в очередь макрозадач, когда будет готова.

Одна задача всегда представляет собой некоторую функцию, которая должна быть выполнена где-то в будущем. Каждая новая задача может быть взята на выполнение только тогда, когда стек вызовов пуст. Задача считается завершенной, когда стек вызовов снова становится пустым.

# Микротаски

Микрозадачи также выполняются по очереди, но, в отличие от макрозадач, все сразу (то есть за одну итерацию цикла событий). Более того, даже если некоторые микрозадачи были добавлены во время выполнения предыдущих, они все равно будут завершены в текущей итерации. Примеры микрозадач: функции обратного вызова для обработки промисов и наблюдателя мутаций.

Порядок выполнения задач заслуживает особого внимания. Цикл событий «заглядывает» в очередь микрозадач после каждого выполнения макрозадачи. То есть, чтобы перейти к следующей макрозадаче, все существующие микрозадачи должны быть завершены.

# Пример использования макро- и микротасок

```
1   console.log('111');
2
3   setTimeout(() => console.log('222'), 0);
4
5   Promise.resolve('333')
6       .then((val) => console.log(val));
7
8   console.log('444');
9
10  Promise.resolve('555')
11      .then((val) => console.log(val));
```

# async/await

Промисы позволили избавиться от callback ада, однако сами породили другую проблему: chaining.

В принципе, chaining не является проблемой на небольших проектах, однако представьте себе такую ситуацию, в которой у нас используется ОЧЕНЬ много .then().

Поэтому, для удобочитаемости были сделаны async/await

# Сравнение

```javascript
// Вспомогательная функция для имитации асинхронного запроса
function delay(ms, data) {
  return new Promise(resolve => setTimeout(() => resolve(data), ms));
}

// Основная асинхронная функция
async function processUserData() {
  try {
    // 1. Получаем пользователя
    const user = await delay(500, { id: 1, name: 'Алексей' });
    console.log(`Пользователь: ${user.name}`);

    // 2. Получаем посты пользователя
    const posts = await delay(600, [
      { id: 101, title: 'Первый пост', userId: user.id },
      { id: 102, title: 'Второй пост', userId: user.id }
    ]);
    console.log(`Получено постов: ${posts.length}`);

    // 3. Берём первый пост и получаем его комментарии
    const firstPost = posts[0];
    const comments = await delay(400, [
      { id: 1001, text: 'Отличный пост!', authorId: 5 },
      { id: 1002, text: 'Спасибо, полезно', authorId: 7 },
      { id: 1003, text: 'Лучший комментарий!', authorId: 3 }
    ]);
    console.log(`Получено комментариев: ${comments.length}`);

    // 4. Находим лучший комментарий (синхронная логика)
    const bestComment = comments.reduce((best, curr) =>
      curr.text.length > best.text.length ? curr : best
    );
    console.log(`Лучший комментарий: "${bestComment.text}" (автор ID: ${bestComment.authorId})`);

    // 5. Получаем данные автора лучшего комментария
    const author = await delay(300, { id: bestComment.authorId, name: 'Екатерина', email: 'katya@example.com' });
    console.log(`Автор лучшего комментария: ${author.name} (${author.email})`);

    console.log('✅ Асинхронная операция успешно завершена!');
  } catch (error) {
    console.error('❌ Произошла ошибка:', error);
  }
}
```

```javascript
// Вспомогательная функция для имитации асинхронного запроса
function delay(ms, data) {
  return new Promise(resolve => setTimeout(() => resolve(data), ms));
}

// Цепочка промисов
delay(500, { id: 1, name: 'Алексей' })  // Получаем пользователя
  .then(user => {
    console.log(`Пользователь: ${user.name}`);
    // Имитируем запрос постов пользователя
    return delay(600, [
      { id: 101, title: 'Первый пост', userId: user.id },
      { id: 102, title: 'Второй пост', userId: user.id }
    ]);
  })
  .then(posts => {
    console.log(`Получено постов: ${posts.length}`);
    const firstPost = posts[0];
    // Имитируем запрос комментариев к первому посту
    return delay(400, [
      { id: 1001, text: 'Отличный пост!', authorId: 5 },
      { id: 1002, text: 'Спасибо, полезно', authorId: 7 },
      { id: 1003, text: 'Лучший комментарий!', authorId: 3 }
    ]);
  })
  .then(comments => {
    console.log(`Получено комментариев: ${comments.length}`);
    // Находим лучший комментарий (допустим, по длине текста)
    const bestComment = comments.reduce((best, curr) =>
      curr.text.length > best.text.length ? curr : best
    );
    console.log(`Лучший комментарий: "${bestComment.text}" (автор ID: ${bestComment.authorId})`);
    // Имитируем запрос информации об авторе лучшего комментария
    return delay(300, { id: bestComment.authorId, name: 'Екатерина', email: 'katya@example.com' });
  })
  .then(author => {
    console.log(`Автор лучшего комментария: ${author.name} (${author.email})`);
    console.log('✅ Цепочка промисов успешно завершена!');
  })
  .catch(error => {
    console.error('❌ Произошла ошибка:', error);
  });
```

# Запросы на сервер

# Немного теории про работу с запросами

Отправка запросов на сервер в JS осуществляется через метод fetch() (современный подход) или XMLHttpRequest (устаревший). fetch() использует промисы, что делает код чище при GET/POST запросах. Данные обычно передаются в формате JSON, а для работы с ответом используются методы .then() или async/await.

# fetch

Fetch API предоставляет интерфейс JavaScript для работы с запросами и ответами HTTP. Он также предоставляет глобальный метод fetch(), который позволяет легко и логично получать ресурсы по сети асинхронно.

Подобная функциональность ранее достигалась с помощью XMLHttpRequest. Fetch представляет собой лучшую альтернативу, которая может быть легко использована другими технологиями, такими как Service Workers. Fetch также обеспечивает единое логическое место для определения других связанных с HTTP понятий, такие как CORS и расширения для HTTP.

# Примеры использования fetch()

```
async function getPosts() {
  try {
    // Делаем запрос
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1');

    // Проверяем, успешен ли ответ (статус 200-299)
    if (!response.ok) {
      throw new Error(`Ошибка HTTP: ${response.status}`);
    }

    // Парсим JSON
    const post = await response.json();
    console.log('Полученный пост:', post);
    return post;
  } catch (error) {
    console.error('Ошибка при запросе:', error.message);
  }
}

getPosts();
```

```
fetch('https://jsonplaceholder.typicode.com/posts/2')
  .then(response => {
    if (!response.ok) {
      throw new Error(`Ошибка HTTP: ${response.status}`);
    }
    return response.json();
  })
  .then(post => {
    console.log('Полученный пост (через then):', post);
  })
  .catch(error => {
    console.error('Ошибка при запросе:', error.message);
  });
```

# Axios

Axios - это HTTP-клиент, основанный на Promise для node.js и браузера. Он изоморфный (= он может работать в браузере и node.js с той же базой кодов). На стороне сервера он использует нативный node.js http-модуль, тогда как на стороне клиента (браузер) он использует XMLHttpRequest.

# Пример кода с использованием axios

```
async function getPost() {
  try {
    const response = await axios.get('https://jsonplaceholder.typicode.com/posts/1');
    console.log('Полученный пост:', response.data);
  } catch (error) {
    console.error('Ошибка:', error.message);
  }
}

getPost();
```

```
axios.get('https://jsonplaceholder.typicode.com/posts/2')
  .then(response => {
    console.log('Полученный пост (через then):', response.data);
  })
  .catch(error => {
    console.error('Ошибка:', error.message);
  });
```

Статьи к ознакомлению:
https://developer.mozilla.org/ru/docs/Web/API/Fetch_API/Using_Fetch
https://habr.com/ru/companies/ruvds/articles/477286/
https://learn.javascript.ru/async - может не зайти без VPN
https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/Promise
https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Statements/async_function
https://developer.mozilla.org/ru/docs/Learn_web_development/Extensions/Async_JS/Promises
https://habr.com/ru/articles/762618/ - MUST HAVE к прочтению